Comandos Linux

Comandos I

- **date** indica la hora del sistema
- **hwclock** hora presente en BIOS
- **date** -d "next friday" **date** -d "tomorrow"
- **date** -d "4 weeks" **date** -d "4 weeks ago"
- **date** -d "3 years ago 4 month ago 3 days ago"
- **date** +%B
- **date** +%F
- **date** +%d%m%y

Comandos

- Instalar paquete ncal
  - apt update
  - apt install ncal
- **cal** **cal** 12 1970 **cal** -3 12 1970
- **ncal** -w muestra también número de la semana

Comandos I

Encadenado de comandos: operador ;

- La ejecución de los comandos es secuencial:
- Se ejecuta primero, comando1, una vez que haya acabado la ejecución de comando1, se ejecutará comando2, y una vez finalizada la ejecución de comando 2 se ejecutará por último comando3
- comando1;comando2;comando3
- date;pwd;cal

Comandos I

Visualizar texto por pantalla:

- Comando **echo**
  - **echo "**Hola Mundo"
  - Para dar formato al texto utilizar opción **-e**
  - **echo** –e "Hola\nme\nllamo\nPepe"
  - \n: Salto de línea. \\: barra
  - \t: Tabulación horizontal.
  - \b: carácter atrás.

Comandos I

Visualizar texto por pantalla:

- Comando **echo**
  - **echo** $variable Muestra el valor de la variable
  - echo $HISTSIZE Muestra el valor de la variable
  -

Comandos I

- **Sintaxis de un comando**
  - **Comando** [**parámetros**] [**argumentos**]
  - Los corchetes indican, que lo que encierran, es opcional, es decir puede estar o no.
  - [**parámetros**] son las opciones del comando. Suelen comenzar con un guión, opción POSIX, o doble guión, opción GNU/Linux, y afectan al comportamiento del comando.
  - [**argumentos**] son los valores en los que un comando debe realizar una acción, ficheros, texto etc.
  - Los comandos pueden tener ningún o varios parámetros y argumentos.

Comandos I

- **Sintaxis de un comando**
  - Los parámetros pueden ponerse de forma individual:
    - ls -l -i -h -a
  - O agrupados:
    - ls -liha
  - Pueden mezclarse el formato corto y largo: ls --all -l
  - Los parámetros no están estandarizados, el mismo parámetro es posible que no tenga el mismo significado en distintos comandos.
    - ls -l aquí tiene el significado de formato largo
    - wc -l aquí cuenta el número de líneas devueltas
    - df -h;ls -h aquí tienen el mismo significado, ¿cuál?

Comandos I

- **Sintaxis de un comando**
  - Los parámetros también puede ir precedidos por doble -, opción GNU/Linux
  - De esta forma, el parámetro es más identificable
    - df -h ó df--human-readable
    - ls -d ó ls --directory
  - Doble guión, - - sin opción, significa final de los parámetros y principio de los argumentos
  - Comando -- y presionas dos veces tabulador, aparecerá todas las opciones válidas. Cosa que no ocurre con un único guión -

Comandos I

- **Comando internos y externos:**
  - Comando externos: son los programas binarios almacenados en HD y cargados en memoria, para poder ejecutarlos, se ejecutan en una nueva shell.
  - Comando internos: son los que se ejecutan dentro de la Shell donde se invocan, forman parte del Shell.
  - **type** comando indica tipo comando
  - **file** path-absoluto_fichero indica tipo fichero

Comandos I

- **Comando internos y externos:**
  - Ciertos comandos tienen una implementación interna y externa:
  - El comando interno se ejecutará con mayor prioridad
  - La ejecución de un comando interno es más rápida.
  - Comando type interpreta si el comando es interno, pero no precisa si hay una implementación externa.
  - **type** pwd
  - **ls** -l /bin/pwd **type** /bin/pwd
  - **type** nombre_función
    - Indica que es una función y muestra su código.

Comandos I

- **Variables:**
  - Guardan en memoria un valor, y son accesibles mediante el nombre asignado a la variable precedidos por $.
  - Creación de una variable:
    - nombre_variable=valor_variable
    - cadena="esto es una cadena"
    - entero=10
  - Visualización del contenido de una variable:
    - echo $cadena
    - echo $entero

Comandos I

- **Variables:**
  - Las variables de la shell por lo general vienen en mayúsculas.
    - echo $HISTFILE
  - echo $ y a continuación pulsamos dos veces el tabulador, veremos todas las variables definidas, tanto las creadas por el usuario en la sesión actual como las definidas por la shell.
  - Para ver el significado de las variables de la shell, man bash y buscar por: Variables de la Shell
  - Las variables tienen que tener un nombre descriptivo de su cometido.

Comandos I

- **Variables:**
  - Asignación del resultado de un comando a una variable.
    - variable=$(comando)
    - fecha=$(date);echo $fecha
    - cores=$(nproc);echo $cores

Comandos I

- **Atajos para comandos:**
  - [Ctrl] **C**: Interrupción del programa, se termina.
  - [Ctrl] **Z**: Para el programa.
    - find /
    - ps --aux
  - [Ctrl] **D**: finalización de archivo,interrumpe introducción de datos en un promt >
  - cat > prueba.txt Introduce texto CTRL D
  - mail

Comandos I

• Histórico de comandos:

- history
- history **n** muestras los últimos n comandos
- Variable $HISTSIZE contiene el número de comandos
- **!!** Vuelve a ejecutar el comando anterior
- **!-n** Ejecuta el comando introducido hace n comandos
- **!$** Muestra el último campo del comando anterior similar a Alt+.
- **!*** Muestra todos los campos del comando anterior
- **!número comando**: ejecuta ese comando.
- !**letra** ejecuta el último comando que comience por letra
- **!?cadena** ejecuta e último comando que contenga cadena
- Con !numero, ver o editar el comando antes de presionar enter: Ctrl+Alt+e ó Ctrl+Esc+e
- **!:0**+Ctrl+Esc+e Muestra el primer campo del último comando introducido.
  - **1**,**2**,.. mostrará segundo, tercero y viceversa.
- Se almacena en ~/.bash_history
- history –c: ¿borra histórico?

Comandos I

- **Las rutas:**
  - El nombre de la ruta o Path name de un fichero es la concatenación, desde la raíz /, de todos los directorios que se deben cruzar para acceder a él, estarán separados cada uno por el carácter /
  - /home/usuario/practicas/pract1.txt
  - Una ruta absoluta no tiene ni . ni ..

Comandos I

- **Ruta relativa:**
  - Partimos de la posición en el directorio actual.
  - Entradas particulares de directorios:
    - **.** Representa el directorio actual. Implícito.
    - **..** Representa el directorio de nivel superior.
  - La ruta relativa:
    - Describe una ruta desde la posición actual.
    - Puede contener . ó ..

Comandos I

- **Ruta relativa:**
  - Documentos/personal
  - **.**/Documentos/personal
    - Donde: **.** Representa el directorio activo
  - /home/usuario/documentos/../

Comandos I

- **Ruta relativa:**
  - Alias del directorio personal: **~** (Virgulilla)
  - Las rutas pueden ser relativas a la virgulilla, pero ésta no debe ir precedida por carácter alguno.
    - cd **~**/docu
    - /**~** Error

Comandos I

- **cd**:
  - Change Directory.
  - cd . No se mueve.
  - cd .. Sube un nivel.
  - cd sin argumento vuelve al directorio personal del usuario.
  - cd ../home
  - cd ../../usuario

Comandos I

- **Listado de ficheros y directorios:**
  - **ls, tree**
    - ls -l es la opción más completa.
    - ls -S ordena por tamaño de mayor a menor
    - ls -t ordena por tiempo primero los más recientes
    - -r invierte el criterio especificado
    - -n uid, gid numéricos
    - -d lista directorios no su contenido
    - -F muestra / final en los directorios
    - -1 muestra una fila por fichero/directorio
    - -R muestra directorio recursivamente
    - -i muestra número de inodo

Comandos I

- **Listado de ficheros y directorios:**
  - **ls, tree**

    Ejercicio:

    Utiliza man ls para ver las opciones.

    Anota las opciones más importantes.

    ¿Qué ficheros son los que comienzan por un punto .?.

Comandos I

- **Listado de ficheros y directorios:**
  - **tree**
    - -u Muestra usuario propietario
    - -g Muestra grupo propietario
    - -p Muestra los permisos
    - -h Muestra tamaños en formato humano
    - --si Muestra tamaños en el Sistema Internacional
    - --du Muestra tamaño del contenido de directorios

Comandos l

```
drwxr-xr-x 5 usuario usuario  4096 oct  3 18:38 pruebas
-rw-r--r-- 1 usuario usuario 62241 oct  3 18:22 resultado
```

1 2 3 4 5 6 7 8

- 1 -: ordinario, d: directorio, l: vínculo simbólico.
- 2 Permisos
- 3 contador de vínculos duros, en caso de ser fichero.En caso de ser un directorio, número de subdirectorios.
- 4 Propietario del fichero que suele ser el creador.
- 5 El grupo al cual pertenece
- 6 El tamaño del fichero en Bytes.
- 7 Fecha de la última modificación.
- 8 Nombre del fichero.

Comandos I

- **Crear ficheros vacíos:**
  - **touch** nombre_fichero1
  - **>** nombre_fichero2
  - **:>** nombre_fichero3
  - Muy práctico para saber si un sistema de archivos está en modo solo lectura.

Comandos I

touch:

- Marcas temporales para archivos:
  - Tiempo de último acceso del archivo, **atime**
  - Tiempo de última modificación del archivo, **mtime**
  - Tiempo de último cambio del archivo, **ctime**
- **El cambio de atime o mtime implica el cambio de ctime.**
- El cambio de los atributos del fichero implica el cambio de ctime, cambio de permisos, propietario, etc.
- touch, asigna la hora actual como hora de modificación y acceso.
- Comando **stat** muestra info de tiempo del fichero

Comandos I

- **touch** [opciones] nombre_fichero1
  - Cambio de hora de acceso,pero no de modificación, a la actual
    - **-a** --time=atime
  - Cambio de hora de modificación, pero no la de acceso, a la actual
    - **-m** --time=mtime
  - Cambio de hora de acceso y modificación, a la actual
    - -**am** --time=atime --time=mtime
  - Asignar fecha especificada, acceso y modificación
    - **-d** marca temporal: "03-feb-2003" ó "03/02/2003"
    - **-d** marca temporal: "21:08" ó "9:08 pm"
    - **-d** marca temporal: "next friday 23:00"
  - Asignar hora especificada, hora acceso y modificación:
    - **-t** marca_temporal
    - Marca temporal: [YY]MMDDhhmm[.ss]

Comandos I

- **touch** [opciones] nombre_fichero1
  - Cambio de hora de acceso y modificación, tomando como referencia la de un fichero dado de referencia :
    - **-r** fichero_referencia
    - touch **-r** fichero_referencia fich1 fich2……
  - Cambio de hora de acceso y modificación mediante enlace simbólico
    - **ln -s** fich1.txt enlace_fich1 enlace_fich1 -> fich1.txt
    - touch -am **enlace_fich1** Cambia marcas temporales a lo apuntado por enlace_fich1
    - touch **-h** enlace_fich1 Cambia marca temporal a enlace_fich1

Comandos I

Crear directorios

- **mkdir** nombre
- **mkdir** dir1 dir2 dir3
- **mkdir** -p prueba/dirt1 prueba/dir2
- **mkdir** -p ./mnt/{dir1,dir2,dir3}
- **mkdir** -m 750 prueba Crea con los permisos 750

Suprime directorios:

- **rmdir**

Comandos I

- **Suprime directorios:**
  - **rmdir** no suprime una estructura.
  - El directorio a eliminar no puede tener ni ficheros ni directorios, aunque los subdirectorios están vacíos.
  - Para suprimir una estructura utilizar:
    - $ **rm**.

Comandos I

- **rm** : Suprime uno o varios fichero y estructuras completas.
- **¡¡ALTAMENTE PELIGROSO!!.**
- **rm** [opciones] fic1 fic2 fic3
- **-i**: Confirmación para cada uno de los ficheros a eliminar.
- **-r**: siguiente parámetro debe ser un directorio. La supresión es recursiva. Se suprimen todos los niveles inferiores.
- **-f**: fuerza la supresión.
- Muy práctico pero peligroso combinar: **-rf**
- No tiene funcionalidad tipo papelera, como ocurre en GUI
- Si se quiere recuperar fichero borrado: **foremost**

Comandos I

- **Copiar ficheros:**
  - **cp** fich1 [fich2 fich3] Destino
  - **-i**: Pide confirmación de escritura para cada fichero.
  - **-r**: recursivo, copia un directorio y todo su contenido. **No conserva** atributos de los archivos. Si conserva enlaces simbólicos. $ cp –r dir1 dir2
  - **-a**: recursivo, copia un directorio y todo su contenido. **Si conserva** atributos de los archivos. Si conserva enlaces simbólicos. $ cp –r dir1 dir2
  - **-p**: se preservan los permisos y las fechas.
  - **-f**: forzar la copia.
  - **-u -- update**: copia el archivo sólo si el original es más reciente.

Comandos I

- **Ejercicio:**
  - Crear un fichero de nombre:-i
  - Tratar de borrarlo.

  - Ayuda: utiliza la opción GNU - - que significa final de los parámetros y principio de los argumentos, ó añadir ruta relativa ./ al fichero.

Comandos I

- **Mover y renombrar ficheros:**
  - **mv**
  - **mv** fich1 fich.old Cambia nombre
  - **mv** fich2 dir/fich2 Mueve fichero
  - **mv** fich2 dir/fich2.old Mueve fichero y cambia nombre
  - **mv** dir1/ dirold Cambia nombre
  - **mv** dir2/ dir1/dir2/ Mueve directorio
  - ¿Qué función tiene el parámetro -u?

Comandos I

- **Mover y renombrar directorios:**
  - **mv** dir1/ dirold Cambia nombre al directorio
  - **mv** dir2/ dir3/ Mueve directorio y su contenido
  - Apreciar la barra final, que aparecerá con el directorio al tabular. Esto evitará posibles errores de escritura
  - ¿Qué función tiene el parámetro -u?

Comandos I

- **Ruta de archivos**
  - Impresión únicamente del nombre de fichero, excluye el path y el sufijo si se especifica.
    - **basename** ruta [sufijo]
    - **basename** /home/usuario/fichero1.txt txt
      - Resultado: fichero1
  - Impresión únicamente del path del archivo, excluye el nombre del fichero.
    - **dirname** ruta
    - **dirname** /home/usuario/fichero1.txt
      - Resultado: /home/usuario

Comandos I

• Expansión en bash

– Las expansiones en Bash son procesos que el shell realiza sobre la línea de comandos para interpretar y reemplazar ciertos elementos antes de que el comando se ejecute.

– Los tipos principales incluyen:

- La expansión de histórico !
- La expansión de alias.
- La expansión de llaves (para generar cadenas de texto)
- La expansión de tildes (para expandir rutas de directorio a partir de ~)
- La expansión de parámetros y variables (para utilizar valores de variables)
- La sustitución de comandos (para usar la salida de otro comando)
- La expansión aritmética (para realizar cálculos)
- La expansión de nombres de fichero (para expandir comodines como *).
- Sustitución de comillas.
- Redirecciones

Comandos I

- **Expansión de nombres de ficheros** (Comodines)
  - Pueden ser utilizados dentro de todos los argumentos que representen ficheros o rutas.
  - Se deben cerrar algunos caracteres especiales.
  - **** permite cerrar un único carácter.
  - ls fichero\ * . Se pone \ para que interprete el carácter en blanco como tal.
  - Las comillas " ", permiten la interpretación de caracteres especiales o de variables, dentro de una cadena.
  - Los apóstrofes ' ' cierran todos los caracteres especiales de una cadena o fichero.

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones básicas:
  - Permite sustituir patrones con comodines (*, ?, [ ]) por nombres de archivos existentes en el sistema de archivos.
  - Son caracteres de sustitución.
  - *: Representa una cadena de longitud variable.
  - **?**: Representa un único carácter.
  - **[**ahw**]**: Representa el carácter o caracteres especificados
  - **[**a-g**]**: Representa un carácter entre la serie o rango de caracteres.
  - **[**!a-g**]**: Representa un carácter fuera de la serie o rango de caracteres.
  - **[^**a-g**]**: Representa un carácter fuera de de la serie o rango de caracteres..
  - **{**a,g,h**}**: Un carácter de los especificados.

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones básicas:
  - ls -d * Muestra ficheros y directorios, no muestra el contenido de los directorios
  - ls -d */ Muestra sólo los directorios sin mostrar su contenido
  - ls ab*
  - ls a[b]*
  - ls [ab]*
  - ls [ab]*.[a-z]
  - ls ?[A-Z0-9i]*
  - ls [!a-z]*
  - ls *.?

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones básicas:
  - Es posible especificar múltiples expresiones en la línea de comandos. Estas expresiones deberán estar separadas por un espacio.
  - mv fich**?**.c fich**?**.txt ../tmp

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones compleja:
  - **?(**Expresión**)**: Expresión estará presente 0 ó 1 vez. ls -l fich**?(**49**)**.txt
  - ***(**Expresión**)**: Expresión estará presente entre 0 y n veces. ls -l fich***(**49**)**.txt
  - **+(**Expresión**)**: Expresión estará presente entre 1 y n veces. ls -l fich**+(**49**)**.txt
  - **@(**Expresión**)**: Expresión estará presente sólo 1 vez. ls -l fich**@(**49**)**.txt
  - **!(**Expresión**)**: Expresión no estará presente. ls -l fich**!(**49**)**.txt

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones compleja:
  - Alternativas: una **|** dentro de una expresión compleja significa "o bien".
  - **?(**Expresión**|**Expresión**|**....**)**
  - ***(**Expresión**|**Expresión**|**.....**)**
  - **+(**Expresión**|**Expresión**|**....**)**
  - **@(**Expresión**|**Expresión**|**...**)**
  - **!(**Expresión**|**Expresión**|**.....**)**

Comandos I

- **Expansión de nombres de ficheros** (Comodines) Expresiones compleja:
  - ls -l !(*.xz)
  - ls -l !(*.xz|*.deb)
  - ls -l fich@(A0|A1).txt
  - ls -l fich@(A[0-9]|B[0-9]).txt
  - ls -l fich@(+(A)|+(B))[0-9].txt

Comandos I

- **Expansión de Comandos**
  - Sintaxis:
    - $(**Comando**) ó `**Comando**`
  - Se sustituye por la ejecución de **Comando**

Comandos I

• Expansión de llaves (bash)

- Mecanismo que permite generar patrones o cadenas de texto.
- Similar a la expansión de nombres de archivos, pero no confundir, ya que la expansión de nombre de ficheros permite sustituir patrones con comodines (*, ?, [ ]) por nombres de archivos existentes en el sistema de archivos.
- echo {a..g} Genera un patrón o cadena de texto a b c d e f g
- echo [a-g] Muestra la cadena [a-g]
- ls -l {a..g}*

  ls: no se puede acceder a 'b*': No existe el fichero o el directorio

  ls: no se puede acceder a 'e*': No existe el fichero o el directorio

  ls: no se puede acceder a 'f*': No existe el fichero o el directorio

  ls: no se puede acceder a 'g*': No existe el fichero o el directorio

  -rw-r--r-- 1 root root 38504 feb 9 2019 apt-file_3.2.2_all.deb

  -rw-r--r-- 1 root root 36 feb 9 2019 conffiles
- mkdir [a-g] genera el directorio '[a-g]'
- mkdir {a..g} mkdir a b c f g genera directorios a b c d e f g

Comandos I

• Expansión de llaves (bash)

- La expansión de nombre de ficheros permite sustituir patrones con comodines (*, ?, [ ]) por nombres de archivos existentes en el sistema de archivos.

- mkdir [a-g]
  - Si [a-g] no coincide con ningún fichero ni directorio existente que sea a b c d e f g, entonces no hay expansión y creará el directorio [a-g].
  - Pero si existe los directorios a b c d e f g, se expandirá mkdir a b c d e f g, y al existir no los creará indicando que ya existen, ya que la expansión se realiza por nombres de archivos existentes en el sistema de archivos.
- chmod 700 [a-g]
  - si existen los ficheros o directorios a b c d e f g, se expandirá chmod a b c d e f g, y al existir asignará los permisos, ya que la expansión se realiza por nombres de archivos existentes en el sistema de archivos.
  - Si no existen no se asignarán permisos, pero si existe el directorio '[a-g]', se le asignarán los permisos al directorio '[a-g]'.

Comandos I

• Expansión de llaves (bash)

- La expansión de nombre de ficheros permite sustituir patrones con comodines (*, ?, [ ]) por nombres de archivos existentes en el sistema de archivos.

- mkdir {a..g}
  - Se expande a mkdir a b c d e f g. Si no existen los directorios los crea y si existen, indica la imposibilidad de crearlos por que ya existen.
- chmod 700 {a..g}
  - Asignará los permisos a los ficheros o directorios en caso de existir, en caso contrario informará de la imposibilidad debido a su no existencia.

Comandos I

• Expansión de llaves (bash)

- Mecanismo que permite generar patrones o cadenas de texto.
- Similar a la expansión de nombres de archivos, pero no confundir, ya que la expansión de nombre de ficheros permite sustituir patrones con comodines (*, ?, [ ]) por nombres de archivos existentes en el sistema de archivos.
- ls -l [a-g]* Muestra los ficheros que comiencen por el rango a-g.

  Muestra ficheros o directorios que existan, y comiencen por a b c d e f g
  - Sólo muestra los ficheros coincidentes. En la expansión de llaves, si no hay ficheros o directorios que existan coincidentes, indicará la no existencia de los mismos.
- Las llaves operan como cualquier cadena
- Sintaxis: **{**x,y,z**}**
- echo {lunes,martes,miércoles,jueves,viernes}.log
- mkdir {old,stable,unstable}

Comandos I

- **Expansión de llaves (bash)**
  - **Sintaxis**: prefijo{x,y,z}
    - echo prefijo{1,2,3} prefijo1 prefijo2 prefijo3
    - echo {1,2,3}sufijo 1sufijo 2sufijo 3sufijo

    - mkdir {old,stable,unstable}; ls -l

      drwxrwxr-x 2 usuario usuario 4096 oct 12 14:21 old
      drwxrwxr-x 2 usuario usuario 4096 oct 12 14:21 stable
      drwxrwxr-x 2 usuario usuario 4096 oct 12 14:21 unstable

Comandos I

- **Expansión de llaves (bash)**
  - **Sintaxis**: {x,y,z}{v,w}
    - touch {a,b,c}{1,2,3,4}

      a1 a2 a3 a4 b1 b2 b3 b4 c1 c2 c3 c4

    - touch fich{A,B,C}{1,2}

      fichA1 fichA2 fichB1 fichB2 fichC1 fichC2

    - touch fich{a,b,c}{.jpg,.png}

      ficha.jpg ficha.png fichb.jpg fichb.png fichc.jpg fichc.png

Comandos I

- **Expansión de llaves (bash)**
  - **Sintaxis**: {x,y,z}.sufijo
    - ls -l fich{a,b,c}.png

      -rw-rw-r-- 1 usuario usuario 0 oct 12 14:27 ficha.png

      -rw-rw-r-- 1 usuario usuario 0 oct 12 14:27 fichb.png

      -rw-rw-r-- 1 usuario usuario 0 oct 12 14:27 fichc.png

    - chmod 700 fich{a,b,c}.png && ls -l *.png

      -rwx------ 1 usuario usuario 0 oct 12 14:27 ficha.png

      -rwx------ 1 usuario usuario 0 oct 12 14:27 fichb.png

      -rwx------ 1 usuario usuario 0 oct 12 14:27 fichc.png

Comandos I

- **Expansión de llaves (bash)**
  - touch fich{a..c}{1..3}

    ficha1 ficha2 ficha3 fichb1 fichb2 fichb3 fichc1 fichc2 fichc3
  - mkdir dir{A..Z..2}

    dirA dirC dirE dirG dirI dirK dirM dirO dirQ dirS dirU dirW dirY

Comandos I

- **Expansión de llaves (bash)**
  - **Sintaxis**: {x..y[..incremento]}
    - echo {001..010..3}

      001 004 007 010

    - touch fich{a{1..8},b{11..18}}

      ficha1 ficha3 ficha5 ficha7 fichb11 fichb13 fichb15 fichb17
      ficha2 ficha4 ficha6 ficha8 fichb12 fichb14 fichb16 fichb18

    - touch test{a..z..10}{1..10..2}

      testa1 testa5 testa9 testk3 testk7 testu1 testu5
      testu9 testa3 testa7 testk1 testk5 testk9 testu3 testu7

Comandos I

- **Expansión de llaves (bash)**
  - **Sintaxis**: {x..y[..incremento]}
    - for i in {a..z};do echo $i;done
      - Muestra valores de a a la z un carácter por línea

Comandos I

- **Caracteres de protección I:**
  - Sirven para hacer perder el significado a los caracteres especiales de la shell.

Caracteres	**Significado**
Tabulación, salto línea, espacio	Separadores de palabras
&	Ejecución en segundo plano
| << < > >>	Tubería, redirecciones
( ) [ ]	Agrupación de comandos
;	Separador de comandos
* ? [ ] ?( ) +( ) *( ) !( ) @( )	Caracteres de generación de nombres. Expansión de nombres
$ ${ }	Valor de una variable
`` $( )	Sustitución de comandos. Expansión de comando
' ' " " \	Caracteres de protección

Comandos I

- **Caracteres de protección II:**
  - Comillas simples **' '**: o apóstrofes eliminan el significado a todos los caracteres especiales de la shell. Las comillas simples deben ser un número par en la línea de comando, ya que si no es así la shell no interpreta el comando como terminado. Mostrará por tanto el shell secundario
    - echo $HOSTNAME debian
    - echo **'**$HOSTNAME**'** $HOSTNAME'
    - echo **'**Mi equipo se llama **'**$HOME**'** >
    - echo **'**Comillas simples "protegen" a las comillas dobles**'**

      Comillas simples "protegen" a las comillas dobles
    - echo **'**El valor de \$PWD es $PWD**'**

      El valor de \$PWD es $PWD

Comandos I

- **Caracteres de protección III:**
  - Carácter \: la barra invertida elimina el significado especial del carácter que le sigue.
  - echo Ficheros del directorio actual: *
  - echo Ficheros del directorio actual: *
  - echo echo La barra invertida \\ protege a las \'

Comandos I

- **Caracteres de protección IV:**
  - Comillas dobles " ": eliminan el significado de todos los caracteres especiales de la shell, excepto **$ ` ` $()** **** y a sí mismas.
  - echo "Well, isn't that \"special\"?"
    - Well, isn't that "special"?
  - echo "Tienes `ls | wc -l` ficheros en `pwd`"

    Tienes 156 ficheros en /dev
  - echo "Tienes $(ls | wc -l) ficheros en $(pwd)"

    Tienes 156 ficheros en /dev
  - echo "EL valor de \$PWD es: $PWD"

    El valor de $PWD es: /dev
  - Para poder ver las variables del sistema utiliza el comando **env**

Comandos I

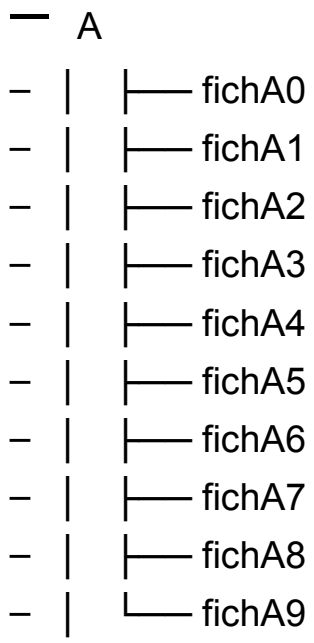
- **Generación de múltiples ficheros I**
  - 1. Crea la siguiente estructura de directorios
    - ├── destino
    - └── origen
    - └── indice
    - └── repositorio
  - 2. Crea en el directorio origen ficheros vacíos de nombre:
    - fichA.txt hasta fichZ.txt
    - ficha.txt hasta fichz.txt
    - fich0.txt hasta fich9.txt
    - A0.txt hasta A9.txt ……Z0.txt hasta Z9.txt
  - ¿Cuántos ficheros has creado?.

Comandos I

- **Generación de múltiples ficheros II**
  - 3. Borra los ficheros fich0.txt hasta fich9.txt. Comprueba que se han borrado.
  - 4. Mueve los ficheros fichA.txt hasta fichZ.txt, al directorio destino. Comprueba que se han movido.
  - 5. Mueve los ficheros ficha.txt hasta fichz.txt, al directorio destino con nombres ficha hasta fichz. Comprueba que se han movido.
  - 6. Crea dentro del directorio indice las carpetas A hasta Z.
  - 7. Mueve A0.txt hasta A9.txt a la carpeta A. Z0.txt hasta Z9.txt a la carpeta Z, de tal manera que en cada carpeta este los ficheros de esa inicial.

Comandos I

- **Generación de múltiples ficheros III**
  - 8. Crea la siguiente estructura de directorios desde A hasta Z en el directorio de nombre repositorio, con una única línea de comando:



- **Generación de múltiples ficheros IV**
  - Solución punto 2 (one-liner):

    - for i in {A..Z};do touch fich$i.txt;done
    - touch fich{A..Z}.txt

    - for i in {a..z};do touch fich$i.txt;done
    - touch fich{a..z}.tx

    - for i in {0..9};do touch fich$i.txt;done
    - touch fich{0..9}.tx

• Generación de múltiples ficheros IV

– Solución punto 2: (Utilizando editor nano)

- Escribe for y pulsa ctrl+x+e, se abrirá nano

  for i in {A..Z};

  do touch fich$i.txt;

  done;

- Pulsa ctrl+x para salir se guarda en un fichero temporal, que será eliminado tras la ejecución.

– Al buscar en el histórico nos aparecerá en formato one-liner en ambas formas.

Comandos I

• Generación de múltiples ficheros IV

– Solución punto 2 (Utilizando editor nano), aprecia que ya no es necesario terminar en ;

- Escribe for y pulsa ctrl+x+e, se abrirá nano

```
for i in {0..9}
do
        touch fich$i.txt
done
```

- Pulsa ctrl+x para salir se guarda en un fichero temporal, que será eliminado tras la ejecución.

– Al buscar en el histórico nos aparecerá en formato one-liner en ambas formas.

Comandos I

Generación de múltiples ficheros IV

- Solución punto 2: (Utilizando editor nano), utilización de sangrado. Más legible.
  - Escribe for y pulsa Ctrl+x+e, se abrirá nano

```
for i in {A..Z}            # for anidados
do
    for j in {0..9}
    do
        touch $i$j.txt
    done
done
```

  - Equivalente:

```
touch {A..Z}{0..9}.txt
ls -l | wc -l
```

Comandos I

- **Generación de múltiples ficheros IV**
  - Solución punto 2: (prompt secundario >)
    - Escribe for i in {0..9} y pulsa intro

      > do

      > touch fich$i

      > done

      pulsa intro para indicar fin, detecta done y finaliza.

    - Al buscar en el histórico nos aparecerá en formato one-liner

Comandos I

- **Generación de múltiples ficheros V**
  - Solución punto 3:
    - for i in {0..9};do rm -r fich$i.txt;done
    - rm -r fich[0-9].txt ó rm -r fich{0..9}.txt ó rm -r fich?.txt
      - ls -l fich[0-9].txt
  - Solución punto 4:
    - for i in {A..Z};do mv fich$i.txt ../destino/fich$i.txt;done
    - mv fich[A-Z].txt ../destino/ ó mv fich{A..Z}.txt ../destino
      - ls -l ../destino/fich[A-Z].txt
    - for i in {a..z};do mv fich$i.txt ../destino/fich$i;done
    - mv fich[a-z].txt ../destino/ ó mv fich{a-z}.txt ../destino/
      - ls -l ../destino/fich[a-z]

- **Generación de múltiples ficheros V**
  - Solución punto 5:
    - for i in {a..z};do mv fich$i.txt ../destino/fich$i;done
    - ~~mv fich[a-z].txt ../destino/fich[a-z]~~
      - mv: cannot stat 'fich[a-z].txt': No such file or directory
    - ~~mv fich{a-z}.txt ../destino/fich{a..z}~~
      - target '../destino/fichz' is not a directory
    - ls -l ../destino/fich[a-z]

Comandos I

- **Generación de múltiples ficheros V**
  - Solución punto 6:
    - for i in {A..Z};do mkdir $i;done
    - ó mkdir {A..Z}
  - Solución punto 7:
    - for i in {A..Z};do for j in {0..9};do mv ij.txt ../indice/$i;done;done

- **Generación de múltiples ficheros V**
  - Solución punto 8:

```
for i in {A..Z}
do
    mkdir $i
    for j in {0..9};
    do
      touch ./$i/fich$i$j.txt;
    done
done
```

Comandos I

- **Concatenación de comandos: ;**
  - Comando1;Comando2;.....;ComandoN
  - Los comandos se ejecutan secuencialmente de izquierda a derecha.

Comandos I

- **Operador Y lógico: &&**
  - Comando1 **&&** Comando2
  - Comando2 se ejecutará si Comando1 se ejecutó con éxito, valor de retorno 0.
    - echo $?
  - La expresión global es verdadera si ambos comandos se ejecutan con éxito.
    - echo $?
  - cd Descargas **&&** ls -l
  - apt-get update **&&** apt-get install htop -y **&&** apt-get clean

Comandos I

- **Operador O lógico: ||**
  - Comando1 || Comando2
  - Comando2 se ejecutará si Comando1 no se ejecutó con éxito, valor de retorno distinto de 0.
  - La expresión global es verdadera si al menos uno de los comandos se ejecutan con éxito.
  - cd Descargas || pwd
  - apt-get install htop -y || apt-get update

Comandos I

- **Agrupación de comandos I:**
  - Se utiliza para dirigir la salida de varios comandos por pantalla hacia el mismo archivo o tubería.
  - Ejecutar múltiples comandos en el mismo entorno.
  - Para la agrupación se puede utilizar:
    - Los paréntesis **( )**. Ejecución en shell hijo
    - Las **{ }**. Ejecución en el shell actual.

Comandos I

- **Agrupación de comandos II:**
  - Sintaxis: (coman1 ; coman2 ; …; comanN)
    - **(**pwd **;** ls**)** > salida.txt
    - pwd;ls > salida.txt
  - Sintaxis: **{** coman1 ; coman2 **;** …**;** comanN **; }**
    - Las aperturas y cierre de llaves deben tener un espacio en blanco, y además el último comando tiene que estar seguido de un espacio y ;
    - **{** pwd **;** ls ; **}** > salida.txt
    - pwd;ls > salida.txt

Comandos I

- **Agrupación de comandos III:**
  - Los paréntesis **( )**. Ejecución en shell hijo
  - Las **{ }**. Ejecución en el shell actual.
  - La ejecución con **{ }** es más rápida, al no tener que crear y ejecutarse en un nuevo shell.
  - La sintaxis con **( )** es más sencilla de recordar

Comandos I

- **Alias**:
  - Comando interno que permite crear atajos a comandos.
  - Creación de alias: **alias** l='ls -l'
  - Visualización de alias: **alias**
  - Visualización de un alias: **alias** l
  - Eliminación de una alias: **unalias** l
  - Para que los alias sean persistentes, introducir línea con la definición del alias en ~/.bashrc

Comandos I

- **Alias**:
  - Creación de alias: **alias** nano='nano -l'
  - type nano nano is aliased to `nano -l'
  - El alias será interpretado antes que el binario nano.
  - Para invocar al binario y no al alias poner \ delante de la llamada al binario nano
  - \nano

Comandos I

- **Alias**:
  - Creación de alias:
    - Tantos alias como definiciones
      - #Alias definidos por el usuario
      - alias nano='nano -l'
      - alias ip="ip a"
    - Agrupación de alias:
      - #Alias definidos por el usuario
      - Alias \
      - up="sudo apt update" \
      - ug="sudo apt update -y && sudo apt upgrade -y"

Comandos I

- **Vínculos Simbólicos (SoftLink):**
  - Es un fichero especial que contiene como información la ruta a otro fichero.
  - Es por tanto como un "Alias".
  - **ln** –s fichero vínculo.
  - Los vínculos se comportan como el fichero al que apunta, con los mismos permisos y propiedades.

Comandos I

- **Vínculos Simbólicos (SoftLink):**
  - Si el fichero apuntado es un programa, ejecutar el vínculo ejecutará el programa.
  - Si el fichero apuntado es un fichero especial, periférico, se ve el vínculo como periférico.
  - Si el fichero apuntado es un directorio, un cd llevará a la entrada en ese directorio.

Comandos I

- **Vínculos Simbólicos (SoftLink):**
  - La eliminación del vínculo no implica la eliminación del fichero apuntado.
  - La eliminación del fichero apuntado no implica la eliminación del vínculo. Vinculo apunta al vacío.
  - Los derechos del fichero apuntado prevalecen sobre los del vínculo.
  - Pueden apuntar a directorios y a otros ficheros de otros sistemas de ficheros

Comandos I

- **Vínculos Simbólicos (SoftLink):**
  - touch prueba1
  - ln –s prueba1 vinculoprueba1
  - ls –li
    - Primer carácter **l** en la salida de ls -l, indicará que es un softlink.
    - **-i** indica el número de inodo
  - ¿Qué derechos tiene tanto el vínculo como el fichero apuntado?.
  - Para saber a que apunta un vinculo:
    - **ls** -l
    - **readlink** nombre_enlace

Comandos I

- **Vínculos (HardLink):**
  - Asocia dos o más ficheros teniendo el mismo inodo.
  - **ln** fichero hardlink
    - touch fichero1
    - ln fichero1 vinculohard
    - ls -li el tercer campo indica el número de enlaces duros
  - Cada enlace duro es una copia exacta del fichero al que apunta, mismo inodo.
  - Si se cambia de ubicación el archivo original, el enlace duro no se rompe.

Comandos I

- **Vínculos (HardLink):**
  - Los enlaces duros ocupan menos tamaño en disco que los enlaces simbólicos.
  - El acceso al contenido a través de un enlace duro, es más rápido que en los enlaces simbólicos.
  - Cuando se realicen cambios en uno de los enlaces o en el fichero este también se realizará en el resto de enlaces.
  - Los enlaces duros no pueden hacerse a directorios y tampoco fuera del propio sistema de ficheros.
  - Usos:
    - Copias de seguridad incrementales, ahorrando espacio y tiempo.
    - Ahorro tiempo en copia de ficheros de gran tamaño.
    - Mediante su uso se pueden tener archivos en varias ubicaciones, evitando que se generen vínculos simbólicos rotos.

Comandos I

- **Vínculos (HardLink):**
  - ls -lhi ó tree –inodes -h

    total **12M**
  - 82954 -rw-r--r-- **2** debian debian 5.7M Nov 16 16:32 HellsBells.mp3
  - 82954 -rw-r--r-- **2** debian debian 5.7M Nov 16 16:32 hard1.mp3

    lrwxrwxrwx 1 debian debian 18 Nov 16 16:37 soft1 ->HellsBells.mp3
  - du -h

    5.7M A todos los efectos HellsBells.mp3 y hard1.mp3 es el mismo fichero
  - ls -lh: Cuenta el número de hard links, por lo que suma el tamaño del archivo tantas veces como hard links existan.
  - du: Cuenta el espacio en disco realmente ocupado, por lo que sólo suma el tamaño del archivo una vez, independientemente del número de hard links.

Comandos I

- **Vínculos (HardLink):**
  - md5sum * Muestra los md5sum de todos los ficheros. Mostrando el mismo valor.
  - find . -inum 82954 -ls encontrará todos los vínculos que llevan el mismo número de inodo

Comandos I

- **Redirecciones**:
  - Capacidad de redireccionar la visualización de la pantalla hacia:
    - Fichero.
    - Impresora.
    - Cualquier otro periférico.
  - Capacidad de redireccionar los mensajes de error hacia otro fichero.
  - Sustituir la introducción vía teclado por un fichero.

Comandos I

- **Redirecciones**:
  - Las redirecciones se utilizan con los comandos.
  - Permiten recuperar el resultado de uno o varios comandos y enviarlo a un archivo, o provocar la lectura de un archivo por un comando.
  - Todo proceso tiene asociado por defecto 3 descriptores o canales.

Comandos I

- **Redirecciones**:
  - Entrada estándar ó descriptor 0 **stdin**.
  - Los procesos que esperan la entrada de información por parte del usuario por la entrada estándar desencadena una solicitud de lectura en el descriptor 0. Si este descriptor está asociado al terminal, asociado por defecto, se asocia al usuario como una petición de lectura del teclado.

Comandos I

- **Redirecciones**:
  - Salida estándar ó descriptor 1 **stdout**.
  - Los procesos que necesitan enviar un mensaje al usuario como resultado de la ejecución de un comando tiene que hacerlo mediante el descriptor 1. Como el descriptor 1 está asociado por defecto a la pantalla, el resultado se mostrará en pantalla.

Comandos I

- **Redirecciones**:
  - Salida de error estándar ó descriptor 2 **stderror**.
  - Los procesos que necesitan enviar un mensaje de error al usuario como resultado de la ejecución de un comando tiene que hacerlo mediante el descriptor 2. Como el descriptor 2 está asociado por defecto a la pantalla, el resultado se mostrará en pantalla.

Comandos I

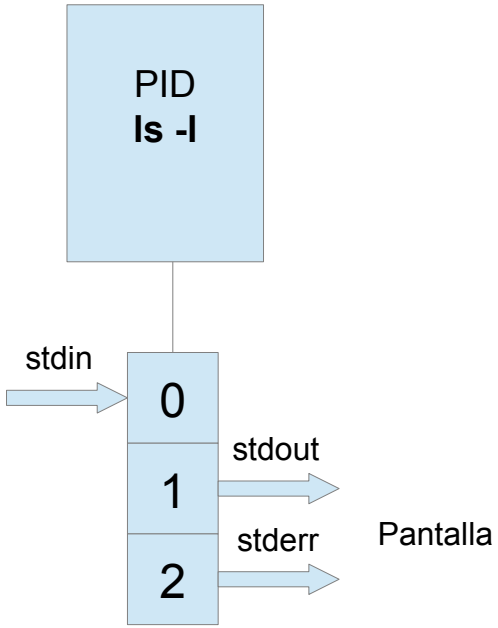
- **Redirecciones**:
  - Por tanto:
    - Cualquier flujo de datos pasa por un canal y podrá ser desviado hacia otro destino u otra fuente.
    - Canal de entrada por defecto es el teclado.
    - Canal de salida por defecto es la pantalla.
    - Los errores se direccionan por defecto a la pantalla.

Comandos I

- **Redirección de salida: > >>**
  - **Redirección simple**: Comando **>** fichero
  - ls –l **>** listado.txt ó ls -l **1>** listado.txt
  - Si no existe listado.txt, se creará.
  - Si existe listado.txt, se sobreescribirá, incluso si el comando introducido fuera incorrecto.
  - El Shell en primer lugar crea el fichero y posteriormente introduce el resultado del comando en listado.txt.

Comandos I

- **Redirección de salida: > >>**



Comandos I

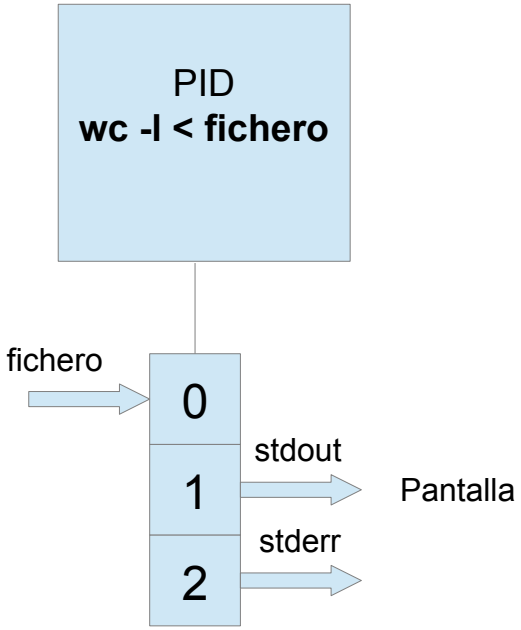
- **Redirección de salida: > >>**
  - **Redirección doble**: Comando **>>** fichero
  - Si lo que se quiere es añadir en el fichero se empleará **>>**
  - ls –l **>>** listado.txt ó ls -l 1**>>** listado.txt
  - De esta forma se añadirán nuevos datos a listado.txt sin sobreescribirlo, por tanto se anexará al final del fichero.
  - ip a **>>** miIP.txt
  - ls -l **2>>** errores.txt

Comandos I

- **Redirección de entrada: <**
  - **Redirección simple**. Los comandos que esperan datos o parámetros pueden también recibirlos desde un fichero usando **<**.
  - cat < ficheros.txt
  - wc **<** fichero.txt ó wc **0<** fichero.txt
  - tr ' ' '\t' **<**prueba.txt
  - mail usuario **<** carta.txt
    - Instala paquetes bsd-mailx y sendmail
    - mail -s Asunto usuario

Comandos I

- **Redirección de entrada: < <<**



Comandos I

- **Redirección de entrada: <<**
  - **Redirección doble**. Herescript o Here Document, permite la inserción de un texto hasta un punto especificado y el envío de su resultado a un comando o un filtro.

```
comando <<DELIMITADOR
línea 1
línea 2
...
DELIMITADOR
```

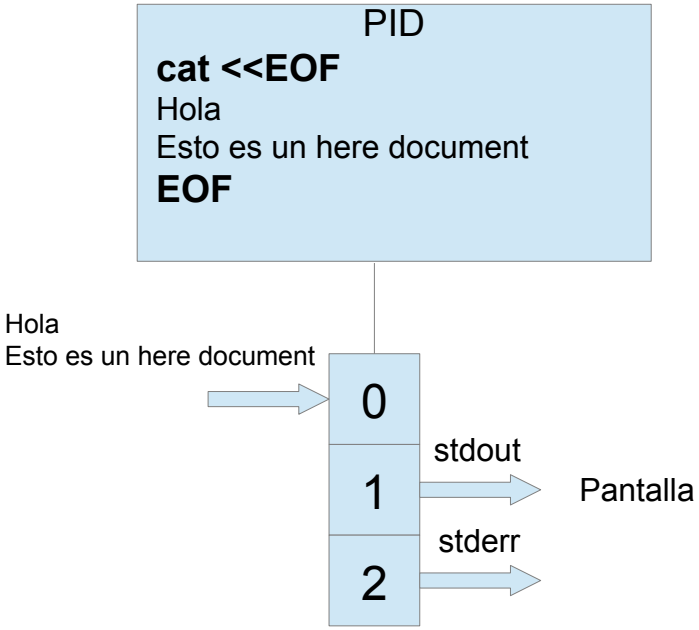
Comandos I

- **Redirección de entrada: <<**

```
cat <<EOF
Hola
Esto es un here document
EOF
cat <<EOF > mensaje.txt
Hola
Esto es un here document
EOF
```

Comandos I

- **Redirección de entrada: < <<**



Comandos I

- **Redirección de entrada: <<**
  - Herescript o Here Document

    tr '[a-z]' '[A-Z]' **<<END**

    >En un lugar de la Mancha.

    >**END**

    wc -w **<<FIN**

    >En un lugar de la Mancha.

    >**FIN**

Comandos I

- **Redirección de entrada: <<**
  - Herescript o Here Document

    tr ' ' '\t' <<END
    - Ahora introducimos un texto. Al terminar de introducir el texto pondremos END, se le pasa al comando tr el texto y sustituirá espacios en blanco por tabulaciones.

Comandos I

- **Redirección de entrada: <<**

  Generación del fichero prueba.txt En formato deb822

  cat <<EOF > prueba.txt

  Types: deb

  URIs: http://repo.cica.es/debian

  Suites: stable

  Components: main

  Architectures: amd64

  Signed-By: /usr/share/keyrings/cica-archive-keyring.gpg

  EOF

Comandos I

- **Redirección de entrada: <<**

  Generación del fichero prueba.txt En formato deb822

  echo "Types: deb" > prueba.txt

  echo "URIs: http://repo.cica.es/debian" >> prueba.txt

  echo "Suites: stable" >> prueba.txt

  echo "Components: main" >> prueba.txt

  echo "Architectures: amd64" >> prueba.txt

  echo "Signed-By: /usr/share/keyrings/cica-archive-keyring.gpg" >> prueba.txt

Comandos I

- **Canales estándares:**
  - Podemos redireccionar los dos canales de salida a un único fichero, utilizando **&>**.

    ls z* y* **>** resultados.log
    - Muestra los errores en pantalla y en resultados.log lo acertado de ls.

    ls z* y* **2>** errores.log
    - Muestra los resultados acertados de ls por pantalla, y se introduce en errores.log los errores de ls.

    ls z* y* **&>** todo.log

    ls z* y* **&>>** todo.log

Comandos I

- **Redirecciones, orden de interpretación por parte del Shell:**
  - Prioridad de caracteres **<,>,>>**
  - Las redirecciones se interpretan de **drcha** a **izq**.
  - Se interpretan las redirecciones **antes** que la ejecución de comandos.

Comandos I

- **Canales estándares:**

  ls z* y* **>** resulconError.log **2>&1**

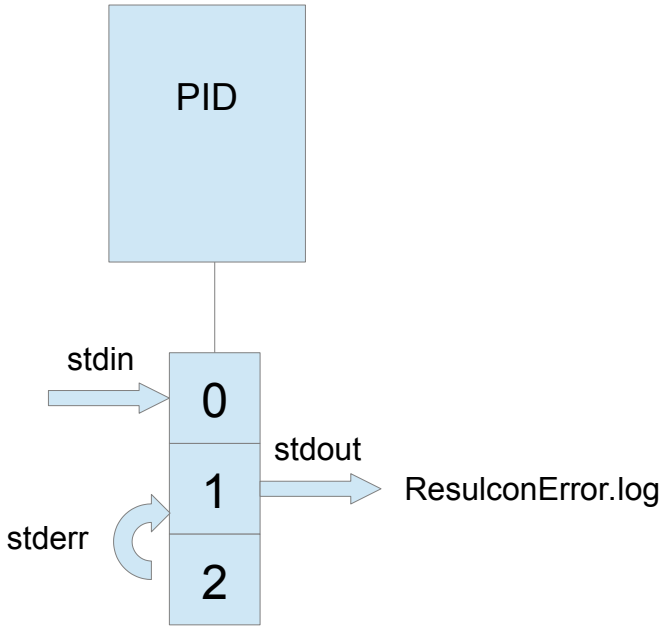
  ls z* y* **1>** resulconError.log **2>&1**

  - En pantalla no se muestra nada estando todos los resultados acertado como errores de ls en resulconError.log
  - En primer lugar se indica que la salida de error se desvíe a la salida estándar, posteriormente se redirecciona el resultado de la salida estándar del comando ls hacia el fichero resultconError.log. El fichero tendrá tanto la salida estándar y la salida de error.

Comandos I

- **Redirección de salida: > >>**

ls z* y* > resulconError.log 2>&1



Comandos I

- **Canales estándares:**
  - ls z* y* **2>** resulconError.log 1**>&2**
    - En pantalla no se muestra nada estando todos los resultados acertado como errores de ls en resulconError.log
    - En primer lugar se indica que la salida estándar se desvíe a la salida de error, posteriormente se redirecciona el resultado de la salida de error del comando ls hacia el fichero resultconError.log. El fichero tendrá tanto la salida estándar y la salida de error.

Comandos I

- **Canales estándares:**
  - time find / > resultado.log
    - Errores en pantalla
    - wc -l **<** resultado.log
    - wc -l **0<** resultado.log
  - time find / 1**>** resulconError.log 2**>&1**
    - Todo al fichero resulconError.log
  - time find / **2>** errores.log
    - Poco efectivo, aciertos por pantalla
  - time find / **2>** errores.log **1>** resul.log
  - wc -l **<** errores.log wc -l **<** result.log

Comandos I

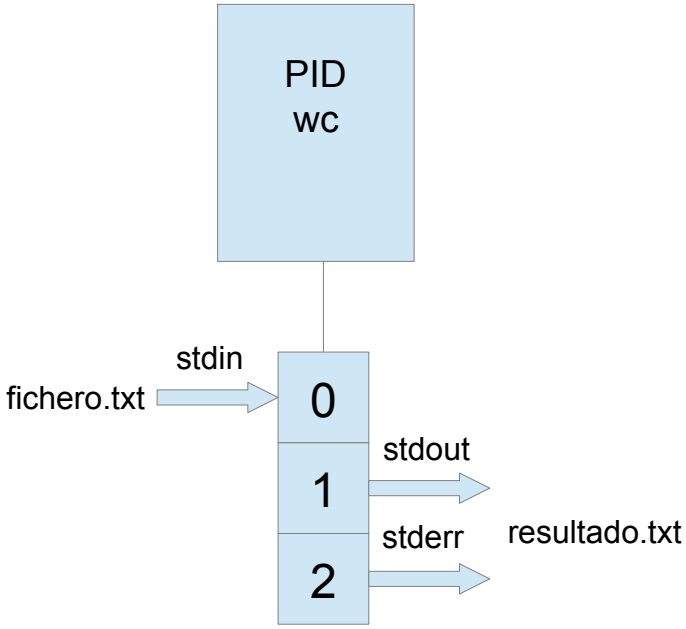
- **Canales estándares:**
  - wc **<** fichero.txt **>** resultado.txt
  - cat resultado.txt
  - ls t* y* **2>**/dev/null
  - ls –l **>** /dev/null 2>&1
  - cat /dev/null

- https://www.ibiblio.org/pub/Linux/docs/LuCaS/Humor/devnull-lleno.txt

Comandos I

- **Canales estándares:**

wc < fichero.txt > resultado.txt



Comandos I

- **Canales estándares:**
  - ¿Cómo saber si un comando lee de su entrada standard?
    - Buscando en el manual del comando.
      - Ver si el argumento file aparece como opcional [file]
      - If no file operands are specified, **the standard input** will be used.
    - Probando el comando sin dar ningún nombre de archivo como argumento. El comando trata a los datos que lleguen por la entrada estándar. Se finalizará la introducción por entrada estándar, teclado, mediante intro y CRTL+d.
    - Ejemplo, comando wc,cat,grep,sort.

Comandos I

- **Canales estándares:**
  - ¿Cómo saber si un comando lee de su entrada standard?
    - Si el comando recibe un fichero como argumento, el comando trabajará con el fichero, y no leerá de la entrada estándar.
    - Si se le pasa como argumento – (guión), el comando leerá de su entrada estándar.
    - grep cadena fichero.txt

Comandos I

- **Apertura de canales:**
  - Los canales estándares son los numerados como 0,1,2.
  - **exec** permite abrir otros siete canales, 3-9.
  - El número de canales en total son del 0-9.
  - Los canales están abiertos para la entrada y para la salida.
  - $exec **5>** canal5.txt #sobreescribe
  - $exec **7>>** canal7.txt #anexa
  - $ls –l **>&5** **>&5** todo junto
  - $cat canal5.txt
  - Todo lo que se escriba por el canal 5 irá a canal5.txt
  - $exec **5>&-** #Cierro el canal 5
  - $exec **7>&-** #Cierro el canal 7

Comandos I

- **Apertura de canales:**
  - $exec **6<** canal6.txt
  - $cat **<&6**
  - $exec **6>&-** #Cerramos el canal.

  Para cerrar el canal lo redireccionamos a un pseudocanal (canal de cierre **-**)
  - Ver descriptores utilizados por un proceso:
    - lsof -p 4565 -d 5
    - 4565 número de proceso asociado a una bash
    - 5 número de descriptor

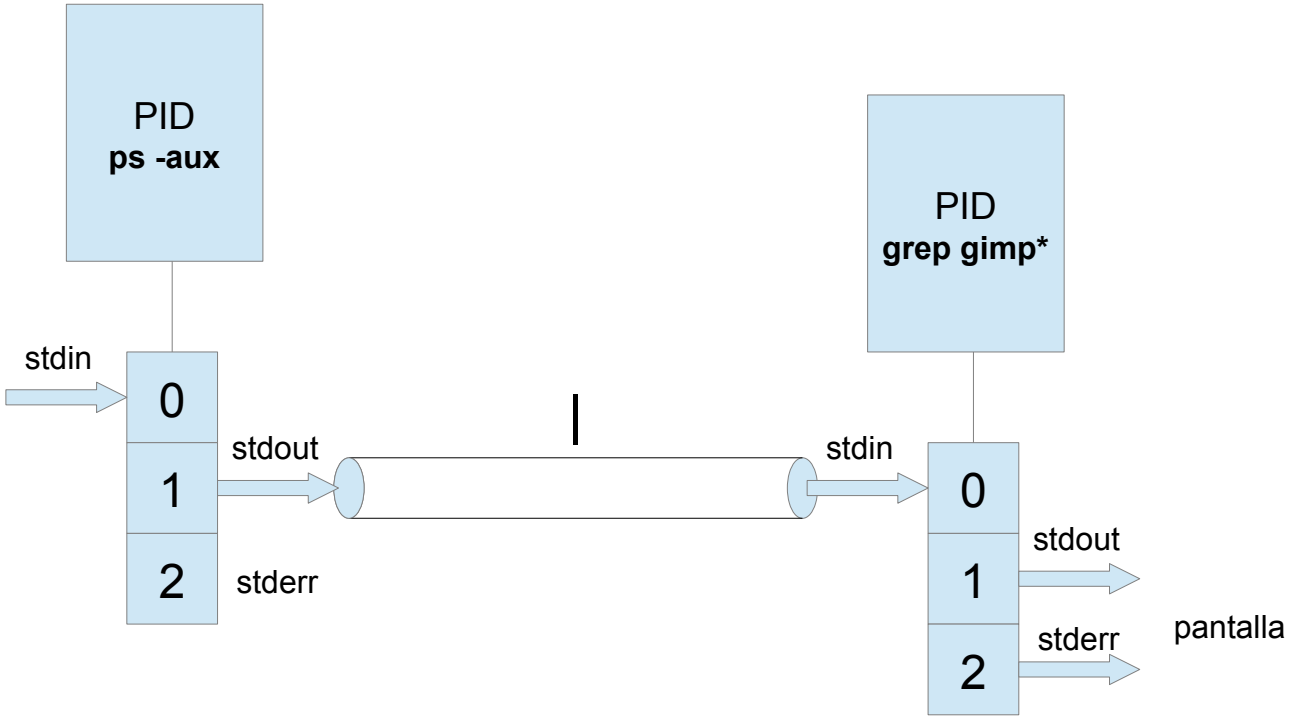
Comandos I

- **Tuberías o Pipelines:**
  - Permiten redireccionar el canal de salida de un comando hacia el canal de entrada de otro comando.
  - Carácter |
  - $ps –aux | grep gimp*
  - $ifconfig eth0 | grep 192.168.1.100
  - Se pueden concatenar múltiples tuberías en la línea de comandos.
  - $ifconfig eth0 | grep 192.168.1.100 | wc -l

Comandos I

- **Tuberías o Pipelines:**

ps –aux | grep gimp*



Comandos I

- **Tuberías con nombre:**
  - Las tuberías anteriormente vistas, se crean, se utilizan y se descartan una vez ejecutado el comando que las utiliza.
  - Las tuberías con nombre son elementos persistente en el sistema de archivos.
  - Son utilizadas en la comunicación entre procesos, para enviar y recibir datos.
  - Los datos permanecerán en la tubería hasta que sean leídos.
  - Poner algo en una tubería con nombre es poner contenido en un extremo de la tubería, el otro extremo de la tubería sólo se abrirá cuando se realice una operación de lectura, es decir extraiga los datos.

Comandos I

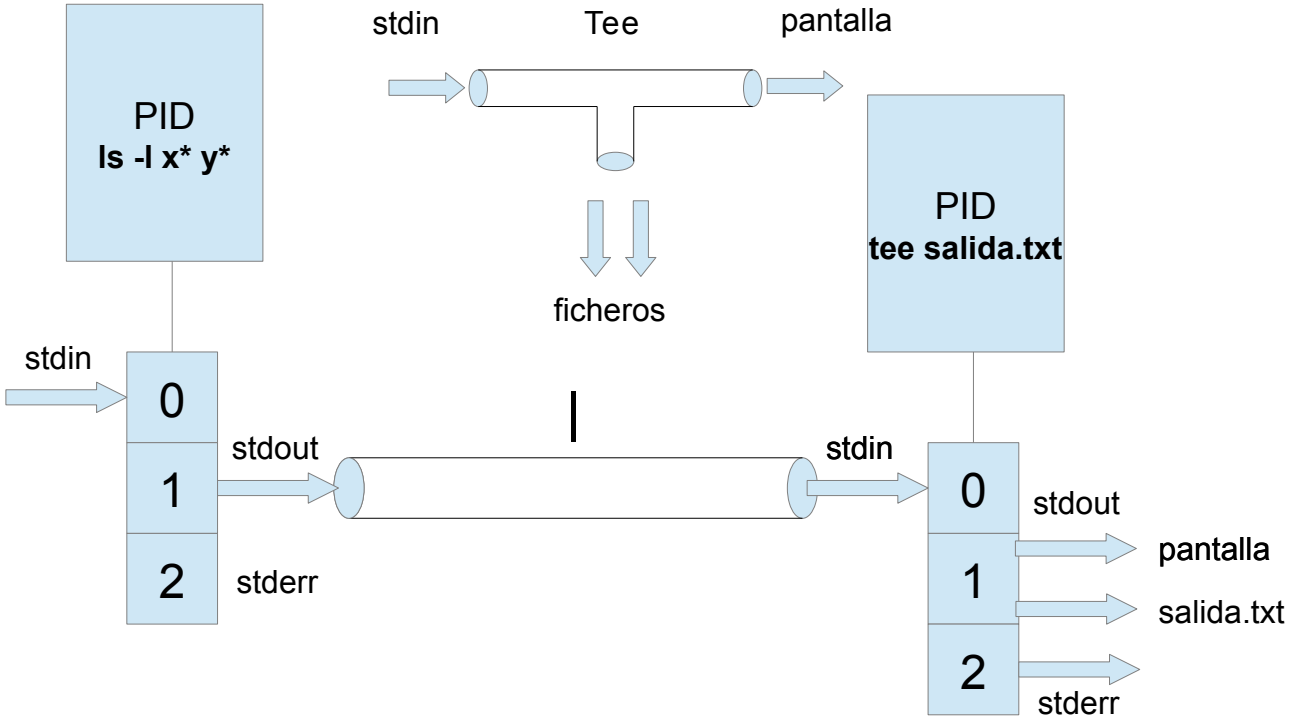
- **Tuberías con nombre:**
  - Sintaxis:
    - mkfifo nombre_tubería
  - ls -l
    - **p**rw-r--r-- 1 rafa rafa 0 dic 6 17:24 **tuberia1**
  - Abrimos dos terminales:
    - En la primera terminal, ejecutamos un comando que canalizaremos a la tubería con nombre. La terminal se quedará a la espera.
      - cat fichero1.txt > tuberia1
    - En la segunda terminal, ejecutaremos el comando que recepcione lo canalizado en la primera terminal.
      - cat < tuberia1

- **Comando tee:**
  - Duplicamos la salida del canal estándar
  - $ tee [-a] fichero
  - $ ls –l | tee midirectorio.txt
    - Muestra por pantalla y crea el fichero con la misma información mostrada por pantalla.
    - Si el fichero no existe lo crea.
    - [-a], permite sobrescribir

Comandos |

- **Comando tee:**

ls -l x* y* | tee salida.txt



Comandos I

- **Comando tee:**
  - $ ls -l x* y*
    - ls: no se puede acceder a 'y*': No existe el fichero o el directorio
    - -rw-r--r-- 1 rafa rafa 0 nov 30 17:32 x1.txt
  - El canal de error del comando situado más al izquierda, no se pasa a la tubería, solamente el canal de salida se pasa a la tubería.
  - $ ls -l x* y* | tee salida.txt
    - ls: no se puede acceder a 'y*': No existe el fichero o el directorio
    - cat salida.txt
    - -rw-r--r-- 1 rafa rafa 0 nov 30 17:32 x1.txt

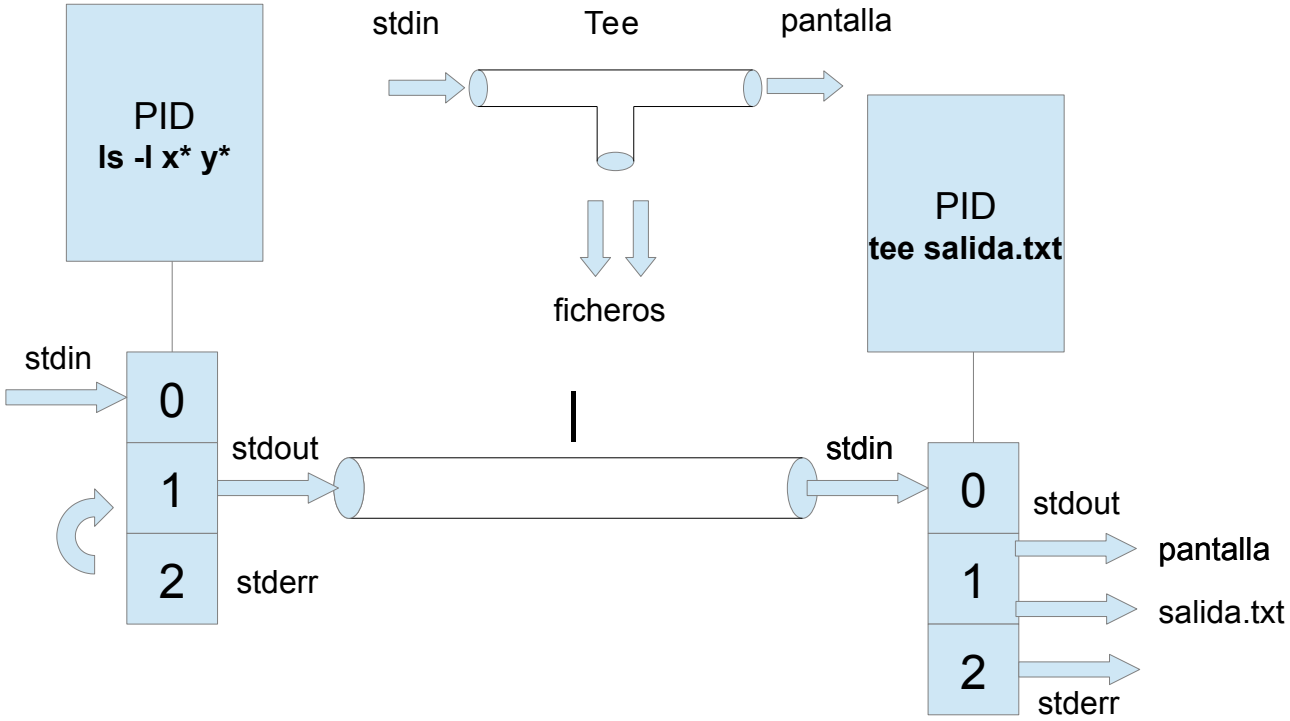
Comandos I

- **Comando tee:**
  - $ ls -l x* y* 2>&1 | tee salida.txt
    - Redirección de canal de error a canal de salida
    - ls: no se puede acceder a 'y*': No existe el fichero o el directorio
    - -rw-r--r-- 1 rafa rafa 0 nov 30 17:32 x1.txt
    - $ cat salida.txt
    - ls: no se puede acceder a 'y*': No existe el fichero o el directorio
    - -rw-r--r-- 1 rafa rafa 0 nov 30 17:32 x1.txt
  - $ ls -l x* y* 2>&1 | tee salida.txt salida1.txt salida2.txt
    - Se almacenará en salida.txt, salida1.txt y salida2.txt tanto el canal de error como el canal de salida.

Comandos |

- **Comando tee:**

ls -l x* y* 2>&1 | tee salida.txt



Comando tee:

- El canal de salida del comando situado más a la izquierda, se pasa a la tubería. Se realiza una escalada de privilegios mediante sudo.
- $ echo "deb http://deb.debian.org/debian buster-backports main" | sudo tee -a /etc/apt/sources.list.d/backports.list

Comandos I

xargs:

- xargs es una utilidad de línea de comandos en sistemas Unix y Linux que lee datos de la entrada estándar (stdin) y los convierte en argumentos para otros comandos. Es decir, toma la salida de un comando y la utiliza como entrada para otro, simplificando la ejecución de tareas en cadena.

Comandos I

xargs:

- En muchos ejemplos introductorios (ls | xargs comando) xargs parece redundante, porque se puede resolver con expansiones de bash (*, $(...), bucles, etc.).

- Hay situaciones reales donde xargs no solo es útil, sino imprescindible o al menos claramente superior, especialmente cuando trabajas con:

  - grandes volúmenes de datos,

    error bash: /bin/rm: Argument list too long
  - nombres de fichero complicados,
  - limitaciones de longitud de línea de comandos, o
  - lectura desde stdin (no desde el sistema de archivos directamente).

Comandos I

xargs:

- Sintaxis:
  - Comando1 | xargs Comando2
  - Comando1 proporciona los argumentos que serán pasados a Comando2
  - xargs acepta datos por su entrada
    - xargs

      Primero

      Segundo

      CTRL+d

      Muestra Primero Segundo en una línea

Comandos I

xargs:

- El comportamiento por defecto de xargs es echo
- ls | xargs

  mostrará por pantalla los nombres de los ficheros pasados por ls

- ls | xargs -t

  Indica que hará xargs con los argumentos pasados por ls

- Muestra en cada línea,5 de los argumentos introducidos

  seq 0 9 | xargs -n5

  0 1 2 3 4

  5 6 7 8 9

Comandos I

xargs:

- fich2.txt

  Primer primero 1 # delimitadores de campos
  Segundo segundo 2 #espacios en blanco o Tab
  Tercero tercero 3
  Cuarto cuarto 4

- cat fich2.txt | xargs -n 6

  Primer primero 1 Segundo segundo 2
  Tercero tercero 3 Cuarto cuarto 4

Comandos I

xargs:

- **-I** {} especifica el valor pasado como argumento y lo reemplaza en {} al comando utilizado por xargs
  - ls | xargs -I {} md5sum {} ó ls | xargs md5sum
  - ls | xargs chmod 600 {} ó ls | xargs chmod
  - Util cuando se utilicen comandos con dos argumentos
- {} puede ser cualquier que no esté reservado @@, ab
  - ls | xargs -I @@ md5sum @@
- **-d** especifica el delimitador de los argumentos introducidos
  - echo "a:b:c:d" | xargs -d ":" a b c d

Comandos I

xargs:

- ls fich 1.png fich 2.png contienen espacios en blanco
- ls fich\ ?.png | xargs **-0**
- find . -iname "fich ?.png" **-print0** | xargs -I {} **-0** gzip {}
- find . -iname "fich ?.png" **-print0** | xargs -0 gzip
- find . -iname "fich ?.png.gz" -print0 | xargs -0 -I {} cp {} gz/{}

Comandos I

xargs:

- **-print0** imprime el nombre completo del archivo en la salida estándar, seguido de un carácter nulo (en lugar de el carácter de nueva línea que utiliza -print). Esto permite nombres de archivos que contienen nuevas líneas u otros tipos de espacios en blanco para que los programas que procesan la salida de búsqueda los interpreten correctamente. La opción corresponde a la opción -0 de xargs.
- **-0** Deshabilita el final de la cadena del archivo, que se trata como cualquier otro argumento. Útil cuando los elementos de entrada pueden contener espacios en blanco, comillas o barras invertidas. La opción GNU find -print0 produce una entrada adecuada para este modo.

Comandos I

xargs: Ejecutar comandos sobre una lista de entrada

- Directorio que contiene las rutas:

  ASIR2/ISO

  ASIR2/LM

  ASIR2/FH

  ASIR2/BD

  ASIR2/Redes

- No es recomendable especificar rutas relativas, ya que no se expanden, se interpreta como caracteres.

  ~/ASIR2/ISO

- En estos casos utilizar variables en la especificación de la ruta a pasar a xargs. centro="IESGN"
- cat directorios3.txt | xargs -n 1 mkdir -p $centro/
- cat directorios3.txt | xargs -I {} mkdir -p $HOME/$centro/{}

Comandos I

xargs: Ejecutar comandos sobre una lista de entrada

- echo -e "acl\nscreen\ntree" | xargs -I {} apt policy {}
- echo "acl screen tree" | xargs -n 1 apt policy
- cat /tmp/paquetes.txt | xargs apt policy
- cat <<FIN | xargs apt policy
  acl
  screen
  tree
  FIN

Comandos I

xargs: Encadenar comandos que no aceptan entrada stdio

- apt policy no acepta datos por su entrada estandar, stdio

- Todas no darán los resutados buscados

  cat /tmp/paquetes.txt > apt policy

  apt policy < /tmp/paquetes.txt

  cat /tmp/paquetes.txt | apt policy

- cat /tmp/paquetes.txt | xargs apt policy

Comandos I

- **xargs**: Ejecutar trabajos en paralelo -P
  - isos_debian.txt
    - https://cdimage.debian.org/debian-cd/current-live/amd64/iso-hybrid/debian-live-12.8.0-amd64-xfce.iso
    - https://cdimage.debian.org/debian-cd/current-live/amd64/iso-hybrid/debian-live-12.8.0-amd64-gnome.iso
    - https://cdimage.debian.org/debian-cd/current-live/amd64/iso-hybrid/debian-live-12.8.0-amd64-kde.iso
  - cat isos_debian.txt | xargs -n 1 **-P 3** -I {} wget {}
  - Se lanzan 3 procesos de descarga que se ejecutarán en paralelo

Comandos I

- **Búsqueda de ficheros:**
  - Comando **find** permite buscar ficheros en el sistema de ficheros mediante criterios.
  - find permite actuar sobre los resultados devueltos.
  - **find ruta criterios opciones**
  - find es recursivo, su punto de inicio es el especificado en ruta.
  - Sitúate en el directorio /tmp
  - find ; find /tmp ¿Qué diferencia encuentras?

Comandos I

- **Búsqueda de ficheros, criterios:**

-name: selección por nombres de ficheros. El criterio debe ir entre comillas dobles. Se pueden utilizar comodines en la especificación de nombre.

find . -name "fich*" -print

- find . -name "p*"
- **-iname**

Comandos I

- **Búsqueda de ficheros, criterios:**

-type: selección por tipo de fichero.

b: Ficheros tipo bloque.

c: Ficheros tipo carácter.

d: dicheros tipo directorio

f: ficheros tipo ordinario

l: ficheros tipo simbólico.

p: ficheros tipo tubería con nombre

s: socket conexión a red.

find -name "p*" -type d -print

Comandos I

- **Búsqueda de ficheros, criterios:**

-user, -group: búsqueda por el propietario o UID. Por grupo de pertenencia del propietario o GUID.

id muestra información sobre UID y GUID.

find . -name "p*" -type d -user 1000 -print

find . -type f -user rafa

Comandos I

- **Búsqueda de ficheros, criterios:**

-**size:** indica el tamaño de los ficheros buscados, por defecto se emplea como unidad el bloque, 512Bytes.

b: Por defecto, bloque de 512 Bytes.

c: Un carácter ASCII, 1 Byte.

w: Una palabra, 2 Bytes.

k,M,G: KibiByte, 1024 Bytes.

+ o – precediendo al valor indican mayor de o menor de.

Comandos I

- **Búsqueda de ficheros, criterios:**

-size 100c busca ficheros de 100 Bytes

-size 100k busca ficheros de 100 kBytes

-size +5k busca ficheros de mas de 5kB.

-size -10k busca ficheros de menos de 10kB.

-size 0 busca ficheros vacíos.

-empty

-size +10M -size -20M busca ficheros de más de 10M y menos de 20M

Comandos I

- **Búsqueda de ficheros, criterios:**

  AND, OR, NOT:

  AND: -a ó **-and** operador **y** lógico, por defecto.

  - $find . -type f -a size -1k -ls

  OR: **-o** ó **-or** operador **o** lógico.

  - $find . -type l -o -type d -ls

  NOT:**-not** ó **!**

  - $find . ! -name "p*" -print

Comandos I

- **Búsqueda de ficheros, criterios:**

stat fichero: muestra info detallada de fichero

atime: fecha de último acceso (access time), puede ser última lectura como último listado ls, cat, grep, less, etc.

mtime: fecha de última modificación (modification time). Modificar contenido, echo, nano, sed, >, etc

ctime: (change time) cambio en el contenido, fecha de última modificación del numero de inodo. Modificación de nombre, cambio de permisos, cambio de propietario o grupo, cambio en el tamaño, creación de enlaces al fichero, etc. Todo lo que suponga modificar metadatos

Comandos I

- **atime, mtime** y **ctime**, trabajan con días (n*24h).

  La referencia es la fecha actual.

  +n busca hace más de n días

  -n busca hace menos de n días

  n busca hace n días

  0 significará el mismo día.

  1 significará ayer, entre 24 y 48 horas.

  2 significará antes de ayer, etc

  mtime 1 ficheros modificados ayer 24-48h

  atime -3 accedidos hace menos de 3 días, es decir 72 horas

  atime +4 ficheros accedidos hace más de 4 días

Comandos I

- **atime, mtime** y **ctime**
- Modificados hace más de 30 días pero menos de 90 días.
  - -mtime +30 -mtime -90
- amin, mmin, cmin
  - Modificados hace más de 120 minutos
    - find . -mmin +120 -type f -ls
  - Modificados hace menos de 120 minutos pero más de 60 minutos
    - find . -mmin -120 -mmin +60 -type f -ls

Comandos I

- **-newer** sirve para comparar marcas de tiempo **entre archivos,** mostrando los ficheros modificados **más recientes** respecto de la fecha del **archivo de referencia**. Hace referencia exclusivamente a modificación.

- archivos más nuevos que la referencia
  - find . -newer referencia1.txt
- archivos **no** más nuevos que la referencia
  - find . ! -newer referencia1.txt
- archivos entre dos fechas o referencias
  - find . -newer referencia1.txt ! -newer referencia2.txt

Comandos I

- **newerat, newremt, newerct**
  - m tiempo de modificación
  - a tiempo de acceso
  - c tiempo de cambio
  - t indica que la referencia se da como tiempo literal, o fechas humanas legibles (time string) en lugar de un archivo
  - fechas humanas legibles:

    "2020-11-01 08:00", "yesterday", "1 week ago"
  - find . -newermt "2020-11-01" ! -newermt "2020-11-30" -ls
  - find ~/ -newermt "last Sunday" -name "*odp"
  - find ~/ -newermt "1 month ago" -name "*odp"
  - find ~/ -newermt "1 week ago" -name "*odp"

Comandos I

- **Búsqueda de ficheros, criterios:**
  - **perm:** búsqueda en las autorizaciones de acceso (derechos, SUID, SGID, Sticky bit).
  - Los derechos se precisan en octal ó en forma simbólica
  - $find . -perm -220 **al menos o como mínimo** esos permisos, **y** lógica
  - find . -perm 664 **exactamente** esos permisos
  - find . -perm -664
  - find . -perm /220 **cualquiera** de esos permisos, **o** lógica
  - find . -perm /u+w,g+w
  - find . -perm /u=w,g=w

  - Busca en el manual referente a find el criterio -perm y vete a la sección de EXAMPLES

Comandos I

- **Búsqueda de ficheros, criterios:**
  - **links:** permite la búsqueda por hardlinks.
  - + ó – más de n vínculos y de menos de n vínculos.
  - **lnum:** permite la búsqueda por número de inodo
  - $find . -inum 82954 -ls encontrará todos los vínculos que llevan el mismo número de inodo.

Comandos I

- **Búsqueda de ficheros, criterios:**
  - $ls -lia i: muestra el numero de inodo
  - $ln Prueba hardlinkaPrueba
  - $ls -lia Prueba nos da número de inodo
  - $stat Prueba nos da número de inodo
  - $find -inum xxxx donde xxxx es el número de inodo, nos mostrará todos los vínculos con el mismo número de inodo que Prueba. Con comando -ls también nos reporta número inodo.

Comandos I

- **Búsqueda de ficheros, comandos:**
  - -**print**: imprime resultados
  - -**ls**: Muestra información detallada de los ficheros encontrados coincidentes con el criterio introducido.
    - $find / -name "Prueba" -ls

Comandos I

- **Búsqueda de ficheros, comandos:**
  - **-exec**: ejecutará el comando colocado a continuación con cada una de las coincidencias que encuentre.
  - find . -type f -name "*.jpg" -exec rm -f {} \;
  - El comando asociado a -exec deberá terminar siempre con \; \escapa a ;
  - {} sustitución de fichero encontrado por find con {} delimitados los argumentos pasados al comando.

Comandos I

- **Búsqueda de ficheros, comandos:**
  - Muy importante, el comando asociado a -exec no esperará a la finalización de find, sino que se aplicará con las coincidencias que vaya encontrando en el proceso de búsqueda.

Comandos I

- **Búsqueda de ficheros, comandos:**
  - ¿Que opción es la más rápida?
  - time find . -type f -name "*.jpg" -exec rm -f {} \;
  - time find . -type f -name "*.jpg" | xargs -I {} rm -f

Comandos I

- **Búsqueda de ficheros, comandos:**
  - **-ok:** realiza la misma acción que -exec, pero pide confirmación para cada coincidencia encontrada.
  - $find . -type f -name "*.jpg" **-ok** rm -f {} \;

Comandos I

- **locate**
  - Búsqueda de ficheros por nombre. No en el sistema de ficheros sino en una Base de Datos de ficheros.
  - Mantiene y consulta un índice de un árbol de directorios.
  - updatedb genera un índice de archivos y directorios. locate de GNU se puede usar para consultar rápidamente dicho índice
  - $ updatedb Actualiza la Base de datos.

Comandos I

- **locate**
  - $ locate fichero
  - -c muestra el número de resultados
  - -i no distingue entre mayúsculas y minúsculas
  - Se puede utilizar comodines y expresiones regulares.
  - locate *.mp3
  - locate --regex '.*duk.mp3$'

Comandos I

- **Búsqueda de ejecutables:**
  - $ whereis nombre
  - $ which comando
  - $ locate fichero
    - locate --regex 'bin/wc$'

Comandos I

- **Búsqueda de ejecutables:**
  - $ whereis nombre
  - Busca en rutas de binarios -b
  - Entradas en el manual -m
  - Fuentes de los ficheros –s